

El Protocolo de Control de Transmisión (TCP) es el estándar de la capa de transporte diseñado para transformar el servicio de entrega de paquetes no orientado a conexión y no confiable de IP (*Best-Effort*) en un servicio de transporte de flujo de datos confiable de extremo a extremo.

1. Características Fundamentales del Servicio

- **Orientado a flujo (Stream):** Los programas de aplicación envían y reciben datos como un flujo continuo y secuencial de octetos.
 - **Conexión de circuito virtual:** Requiere que ambas aplicaciones acuerden y establezcan explícitamente una conexión lógica antes de iniciar la transferencia.
 - **Transferencia con almacenamiento en buffer:** Acumula octetos de datos en buffers para empaquetarlos en segmentos de tamaño eficiente. Las aplicaciones pueden usar el mecanismo *push* (activando el bit PSH) para forzar el envío y la entrega inmediata de los datos acumulados.
 - **Flujo no estructurado:** No impone marcas ni límites de registros en los datos transmitidos.
 - **Comunicación Full Duplex:** Permite la transferencia simultánea y concurrente en ambas direcciones. Además, utiliza *piggybacking* para enviar reconocimientos e información de control adjunta en los segmentos que transportan datos.
-

2. Puntos Finales y Abstracción de Conexiones

- **Ubicación arquitectónica:** Se sitúa en la capa de transporte justo por encima de IP y actúa como par conceptual de UDP.
 - **Definición de punto final (Endpoint):** Un punto final TCP está definido por la tupla (**Dirección IP, Puerto TCP**).
 - **Abstracción basada en conexiones:** A diferencia de UDP (donde los puertos se asocian a colas de mensajes individuales), la abstracción fundamental de TCP es la **conexión**, identificada por un par de puntos finales (Origen, Destino). Esto permite que **un mismo número de puerto TCP local sea compartido por múltiples conexiones simultáneas** en la misma máquina sin ambigüedad.
-

3. Confiabilidad y Mecanismo de Ventana Deslizante

- **Confirmación Positiva con Retransmisión (PAR):** El receptor envía un mensaje de confirmación (ACK) por los datos recibidos con éxito. Si el temporizador del emisor expira antes de recibir el ACK, los datos se retransmiten.
 - **Ventana Deslizante de Octetos:** Para optimizar el rendimiento y evitar la ineficiencia del protocolo "enviar y esperar", TCP permite mantener múltiples octetos "en vuelo" antes de recibir confirmación.
 - **Granularidad y ACK Acumulativo:** Los números de secuencia se asignan a **octetos individuales** en el flujo. Las confirmaciones son **acumulativas** e indican el número de secuencia del *siguiente octeto contiguo* que el receptor espera recibir.
-

4. Control de Flujo y Control de Congestión

- **Control de flujo extremo a extremo:** El receptor especifica su espacio de almacenamiento disponible mediante el campo de anuncio de ventana (*window advertisement*). Si el buffer se llena, el receptor anuncia una ventana de tamaño cero para pausar al emisor.
 - **Control de congestión en la red:**
 - TCP restringe la transmisión al mínimo entre la ventana anunciada por el receptor y su propia **ventana de congestión** (*congestion_window*).
 - **Disminución Multiplicativa:** Reduce la ventana de congestión a la mitad ante la pérdida de un paquete.
 - **Inicio Lento (Slow-Start):** Inicializa la ventana de congestión en 1 segmento y la incrementa en 1 por cada ACK recibido para evitar saturar la red.
 - **AIMD (Incremento Aditivo, Disminución Multiplicativa):** Ajuste continuo de la ventana durante la fase de evitación de congestión.
 - **Retransmisión Rápida y Recuperación Rápida (Reno / NewReno):** Interpreta la llegada de 3 ACKs duplicados como pérdida de paquete para retransmitir inmediatamente sin esperar la expiración del temporizador.
 - **Algoritmo de Karn y Retroceso Exponencial:** Evita mediciones erróneas de RTT al ignorar muestras de segmentos retransmitidos (*ambigüedad de confirmación*) y multiplica el tiempo de espera (*timer backoff*) en cada retransmisión sucesiva.
-

5. Estructura del Segmento TCP y Opciones

- **Encabezado TCP:** Consta de al menos 20 octetos. Incluye:
 - Puertos de Origen y Destino (16 bits cada uno).
 - Número de Secuencia y Número de Confirmación (32 bits cada uno).
 - Longitud del Encabezado (*HLEN*) y Bits de Código (*CODE BITS*: URG, ACK, PSH, RST, SYN, FIN).
 - Tamaño de Ventana y Checksum de 16 bits (calculado incluyendo un pseudo-encabezado IP).
 - **Opciones destacadas:**
 - **MSS (Maximum Segment Size):** Negocia el tamaño máximo del segmento para alinearse con el MTU y evitar la fragmentación en el nivel IP.
 - **Escalado de Ventana (Window Scale):** Aplica un desplazamiento binario al campo de ventana para soportar ventanas mayores a 64 KB en redes de alto rendimiento (*long fat pipes*).
 - **Marca de Tiempo (Timestamp):** Permite medir con precisión el RTT y protege contra la envoltura de números de secuencia (PAWS).
 - **SACK (Selective ACK):** Permite al receptor informar sobre bloques específicos no contiguos recibidos para retransmitir únicamente los datos faltantes.
-

6. Ciclo de Vida de la Conexión

- **Establecimiento:** Utiliza un **intercambio de tres vías (Three-Way Handshake)** con segmentos SYN y ACK para acordar la conexión y sincronizar números de secuencia iniciales aleatorios.
- **Aperturas Pasiva y Activa:** Un extremo realiza una apertura pasiva (esperando peticiones en un puerto) y el otro realiza una apertura activa.
- **Cierre Ordenado:** Finalización simétrica en cada dirección mediante un protocolo de tres pasos modificado con el bit FIN.
- **Reinicio (Reset):** Interrupción abrupta de la conexión enviando el bit RST frente a condiciones anormales.

- **Máquina de Estados Finitos:** Gestiona la evolución de la conexión a través de estados (CLOSED, LISTEN, SYN SENT, SYN RECV, ESTABLISHED, FIN WAIT, TIMED WAIT, etc.). En el estado TIMED WAIT, TCP conserva el registro durante ($2 \times MSL$) (Maximum Segment Lifetime) para asegurar que no queden segmentos antiguos circulando en la red.

Discuss what these sources say about Service Properties, in the larger context of Transmission Control Protocol (TCP).

En el nivel más bajo de la arquitectura de red, las redes físicas y el Protocolo de Internet (IP) proporcionan un servicio de entrega de paquetes **no orientado a conexión y no confiable** (*Best-Effort*), donde los paquetes pueden perderse, retrasarse, duplicarse o entregarse desordenados. Para evitar que cada programa de aplicación tenga que diseñar e implementar complejos mecanismos de detección y recuperación de errores, el **Protocolo de Control de Transmisión (TCP)** ofrece un servicio abstracto de transporte de flujo de datos altamente confiable de propósito general.

1. Las Cinco Propiedades Fundamentales del Servicio TCP

Las fuentes caracterizan el servicio de entrega confiable de TCP mediante **cinco propiedades o características clave**:

1. Orientado a flujo (*Stream-Oriented*)

- **Descripción:** Cuando dos aplicaciones utilizan TCP para transferir datos, la información se percibe y procesa como un **flujo continuo de octetos (bytes)**.
- **Garantía:** La aplicación en el host de destino recibe exactamente la misma secuencia continua de octetos que fue enviada por la aplicación de origen, manteniendo un orden estricto.

2. Conexión de circuito virtual (*Virtual Circuit Connection*)

- **Mecanismo previo:** Antes de iniciar cualquier transferencia de datos, los programas emisor y receptor deben acordar y establecer explícitamente una **conexión TCP** mediante el intercambio de mensajes de control.
- **Abstracción:** Aunque los datos viajan internamente fragmentados en paquetes individuales sobre la red IP, la conexión se comporta conceptualmente ante la aplicación como si existiera un **circuito de hardware dedicado y permanente** entre ambas computadoras.
- **Supervisión:** TCP monitorea continuamente el estado de la comunicación; si el hardware o la ruta de red fallan de forma irrecuperable, se notifica inmediatamente a las aplicaciones asociadas.

3. Transferencia con almacenamiento en búfer (*Buffered Transfer*)

- **Gestión de bloques:** Las aplicaciones entregan datos a TCP en bloques de cualquier tamaño que les resulte conveniente (incluso de un solo octeto). TCP almacena estos octetos en búferes internos para acumular una cantidad suficiente de datos y construir un datagrama razonablemente grande antes de transmitirlo, maximizando así la eficiencia del tráfico de red.
- **Independencia de empaquetado:** TCP tiene total libertad para dividir o agrupar el flujo continuo en paquetes, independientemente de los tamaños de bloque pasados por la aplicación. En el destino, el TCP receptor reensambla los datos en su secuencia original.

- **Mecanismo de empuje (*Push*)**: Para aplicaciones interactivas donde los datos no deben esperar a llenar un búfer (por ejemplo, sesiones de escritorio remoto o terminales interactivas), TCP proporciona la función *push* (activando el bit PSH en el segmento). Esto fuerza al software emisor a transmitir los datos acumulados sin demora y al receptor a ponerlos inmediatamente a disposición de la aplicación de destino.

4. Flujo no estructurado (*Unstructured Stream*)

- **Sin límites de registro**: El servicio de flujo de TCP **no impone ninguna estructura ni marca límites** en los datos transmitidos (por ejemplo, no reconoce dónde empieza o termina un registro dentro del flujo).
- **Responsabilidad de la aplicación**: Son las propias aplicaciones cliente y servidor las que deben ponerse de acuerdo previamente sobre el formato y significado de los datos dentro del flujo continuo para interpretarlo correctamente.

5. Comunicación Full Duplex

- **Transferencia simultánea**: Una conexión TCP permite la transmisión de datos concurrente y simultánea en ambas direcciones. Se compone conceptualmente de dos flujos de datos independientes que viajan en sentidos opuestos sin interferir entre sí.
- **Conexión Half Duplex opcional**: Una aplicación puede cerrar la transmisión en su sentido mientras continúa recibiendo datos del otro extremo.
- **Aprovechamiento de *Piggybacking***: La naturaleza Full Duplex permite que la información de control de un flujo (como confirmaciones de recepción e información de ventana) viaje adjunta en los mismos segmentos que transportan datos en la dirección opuesta, lo que reduce drásticamente la sobrecarga de tráfico en la red.

2. El Contexto General de TCP: ¿Cómo se Logran estas Propiedades?

Para sostener estas cinco propiedades sobre una infraestructura subyacente IP que no es confiable, TCP implementa un conjunto de mecanismos clave:

- **Confirmación Positiva con Retransmisión (PAR)**: El receptor envía mensajes de confirmación acumulativa (ACK) indicando el número de secuencia del siguiente octeto que espera recibir. Si el temporizador del emisor expira antes de recibir el ACK, los datos se retransmiten.
- **Ventana Deslizante a Nivel de Octeto**: A diferencia de los protocolos simples de enviar y esperar, TCP utiliza una ventana deslizante de octetos que permite mantener múltiples segmentos "en vuelo" simultáneamente, saturando eficientemente el canal de comunicación.
- **Control de Flujo Extremo a Extremo**: El receptor informa continuamente su espacio libre en búfer mediante el anuncio de ventana (*window advertisement*) en el campo WINDOW de los segmentos. Si el búfer se llena, el receptor anuncia una ventana de tamaño cero, lo que pausa temporalmente la transmisión del emisor.
- **Control de Congestión**: TCP ajusta activamente la tasa de transmisión al mínimo entre la ventana anunciada por el receptor y su propia **ventana de congestión** (*congestion_window*), reduciendo la ventana a la mitad ante pérdidas de paquetes (Disminución Multiplicativa) y recuperándose gradualmente con Inicio Lento (*Slow-Start*) y Evitación de Congestión (AIMD) para prevenir el colapso por congestión en la red.
- **Puntos Finales y Abstracción de Conexiones**: Las conexiones se definen mediante la tupla de puntos finales (**Dirección IP Origen, Puerto TCP Origen, Dirección IP Destino, Puerto TCP Destino**). Esto permite que múltiples conexiones independientes compartan simultáneamente un mismo puerto local en un servidor sin ambigüedad.

- **Ciclo de Vida de la Conexión:** Las conexiones se establecen mediante un **intercambio de tres vías (Three-Way Handshake)** utilizando bits SYN para sincronizar los números de secuencia iniciales, y se cierran de forma ordenada mediante segmentos FIN o abrupta con el bit RST.
-

Discuss what these sources say about Sliding Window Paradigm, in the larger context of Transmission Control Protocol (TCP).

En los sistemas de comunicación de datos, los protocolos de confirmación positiva más simples (como el esquema **enviar-y-esperar** o *send-and-wait*) obligan al emisor a transmitir un único paquete y detenerse a esperar su mensaje de confirmación (ACK) antes de enviar el siguiente. En redes con retrasos de transmisión o tiempos de ida y vuelta (RTT) elevados, este comportamiento provoca que la red permanezca inactiva la mayor parte del tiempo, desperdiciando capacidad y reduciendo severamente el rendimiento (*throughput*).

El **paradigma de la ventana deslizante** se introduce para resolver este problema, sirviendo como el mecanismo fundamental sobre el que se construye el servicio de transporte confiable de **TCP (Transmission Control Protocol)**.

1. Concepto General de la Ventana Deslizante

- **Transmisiones "en vuelo":** La idea clave es permitir que el emisor mantenga **múltiples paquetes o segmentos transmitidos sin confirmar** (*unacknowledged*) simultáneamente en la red antes de detenerse a esperar respuesta.
 - **Estructura y límites:** Se coloca conceptualmente una "ventana" sobre la secuencia de datos. La ventana divide la secuencia en tres conjuntos:
 1. Octetos/datos a la izquierda: ya transmitidos, recibidos y confirmados con éxito.
 2. Octetos/datos dentro de la ventana: en proceso de transmisión o pendientes de confirmación.
 3. Octetos/datos a la derecha: fuera del límite permitido, los cuales no pueden enviarse hasta que la ventana se mueva.
 - **Desplazamiento contiguo:** A medida que el receptor envía confirmaciones para los elementos del extremo izquierdo, la ventana se **desliza hacia adelante**, permitiendo la transmisión inmediata de nuevos datos.
 - **Saturación óptima:** Un protocolo de ventana deslizante bien ajustado elimina los tiempos muertos, manteniendo el canal de comunicación saturado de datos y alcanzando un rendimiento significativamente superior al de un protocolo simple de confirmación positiva.
-

2. Operación de la Ventana Deslizante en TCP

A diferencia de las descripciones conceptuales simplificadas que operan a nivel de paquetes o tramas, el mecanismo de ventana deslizante de TCP posee características técnicas particulares:

- **Operación a nivel de octetos:** La ventana de TCP no cuenta paquetes, sino **octetos (bytes) individuales** dentro del flujo continuo de datos, los cuales están numerados secuencialmente.
- **Gestión de tres punteros en el emisor:** El emisor mantiene tres marcadores clave para cada conexión:

1. Un primer puntero en el extremo izquierdo que separa los octetos enviados y confirmados de los enviados aún no confirmados.
2. Un segundo puntero que marca el límite de los datos que ya han sido transmitidos pero que aún no reciben ACK.
3. Un tercer puntero en el extremo derecho que define el número de secuencia más alto permitido para transmitir antes de requerir nuevas confirmaciones.

- **Dos ventanas por conexión (Full Duplex):** Dado que TCP proporciona comunicación bidireccional simultánea (*full duplex*), el software de protocolo gestiona **dos ventanas independientes por conexión**: una para monitorear el flujo de datos enviados y otra para estructurar la recepción del flujo entrante.
-

3. Control de Flujo con Tamaño de Ventana Variable

Mientras que los modelos teóricos de ventana suelen asumir un tamaño fijo, TCP implementa un **tamaño de ventana variable** para resolver el problema del control de flujo extremo a extremo entre equipos con diferentes capacidades de procesamiento y memoria:

- **Anuncio de Ventana (*Window Advertisement*):** Cada segmento TCP incluye en su encabezado un campo de 16 bits denominado WINDOW. Mediante este campo, el receptor le informa al emisor la cantidad exacta de octetos adicionales que está dispuesto a aceptar en sus búferes internos.
 - **Ajuste dinámico:** Si los búferes del receptor comienzan a llenarse, este envía anuncios con tamaños de ventana más reducidos, lo que obliga al emisor a estrechar su ventana y reducir la tasa de envío.
 - **Ventana Cero:** Si el búfer del receptor se llena completamente, anuncia un tamaño de ventana igual a cero ($WINDOW = 0$), pausando por completo la transmisión de datos del emisor hasta que se libere almacenamiento y se envíe un nuevo anuncio con ventana mayor a cero.
-

4. Integración con el Control de Congestión en la Red

Además de regular el flujo para no desbordar al receptor, TCP utiliza la estructura de la ventana deslizante para no saturar los enrutadores intermedios de la red (control de congestión):

- **Doble Límite de Ventana:** El emisor gestiona internamente una **ventana de congestión** (*congestion_window*). La ventana efectiva permitida para transmitir (*Allowed window*) se calcula como el mínimo estricto entre el valor anunciado por el receptor y la ventana de congestión impuesta por la red:
[Ventana Permitida = $\min(\text{Anuncio del Receptor}, \text{Ventana de Congestión})$]
 - **Regulación Dinámica (AIMD e Inicio Lento):** Ante la pérdida de paquetes (indicador de congestión), TCP reduce la ventana de congestión a la mitad (**Disminución Multiplicativa**). Posteriormente, al reanudar el tráfico, escala la transmisión partiendo de una ventana de 1 segmento e incrementándola gradualmente (**Inicio Lento / Evitación de Congestión**).
-

5. Confirmaciones y Opciones de Ventana

- **Confirmación Acumulativa:** Los ACKs de TCP indican el número de secuencia del *siguiente octeto contiguo* que el receptor espera recibir. Si se pierde un segmento intermedio dentro de la ventana, las confirmaciones repetidas señalan el punto de interrupción del flujo contiguo.
 - **Confirmación Selectiva (SACK):** Para evitar la ineficiencia de retransmitir toda la ventana cuando solo se pierde un segmento aislado, TCP admite la opción **SACK (Selective ACK)**, la cual permite al receptor notificar explícitamente los bloques no contiguos de datos que sí llegaron con éxito.
 - **Opción de Escalado de Ventana (Window Scale):** Como el campo original WINDOW tiene 16 bits, limita la ventana máxima a 64 KB. En enlaces de alta velocidad con retardo elevado (*long fat pipes*), se utiliza la opción de escalado para aplicar un desplazamiento de bits (*bit-shift*) al campo de ventana, permitiendo tamaños de ventana considerablemente mayores.
-

Discuss what these sources say about TCP Segment Structure, in the larger context of Transmission Control Protocol (TCP).

1. Formato y Campos del Encabezado TCP

Un segmento TCP consta de dos partes principales: un **encabezado TCP** que contiene metadatos de control y un área de **carga útil (payload)**. El encabezado ocupa un tamaño mínimo de **20 octetos (bytes)**, el cual puede incrementarse si se incluyen opciones.

- **Puertos de Origen y Destino (SOURCE PORT y DESTINATION PORT):**
 - Ocupan **16 bits cada uno** y especifican los números de puerto que identifican a los programas de aplicación en los puntos finales de la conexión.
- **Número de Secuencia (SEQUENCE NUMBER):**
 - Campo de **32 bits** que especifica la posición exacta, dentro del flujo continuo de octetos del emisor, que corresponde al primer octeto de datos transportado en el segmento.
- **Número de Confirmación (ACKNOWLEDGEMENT NUMBER):**
 - Campo de **32 bits** que indica el número de secuencia del **siguiente octeto contiguo que el emisor del segmento espera recibir** en el flujo de dirección opuesta.
- **Longitud del Encabezado (HLEN):**
 - Especifica la longitud total del encabezado TCP medida en múltiplos de palabras de 32 bits. Es indispensable debido a que el campo de opciones es de longitud variable.
- **Reservado (RESERVED):**
 - Campo de **6 bits** reservado para uso futuro o funcionalidades avanzadas (como la Notificación Explícita de Congestión, ECN).
- **Bits de Código o Banderas (CODE BITS):**
 - Conjunto de **6 bits de control** que definen el propósito del segmento y cómo deben interpretarse los demás campos:
 - **URG:** Indica que el campo *Urgent Pointer* es válido y hay datos urgentes.
 - **ACK:** Indica que el campo *Acknowledgement Number* contiene una confirmación válida.

- **PSH**: Solicita una entrega inmediata de los datos acumulados (*push*) tanto al emisor como al receptor.
 - **RST**: Solicita el **reinicio (reset)** inmediato y abortivo de la conexión ante errores graves o anormales.
 - **SYN**: Sincroniza los números de secuencia iniciales durante el establecimiento de la conexión (*three-way handshake*).
 - **FIN**: Notifica que el emisor ha alcanzado el final de su flujo de octetos para cerrar ordenadamente la conexión en esa dirección.
- **Ventana (WINDOW)**:
 - Campo de **16 bits** en orden estándar de red donde el receptor anuncia la cantidad de octetos adicionales que está dispuesto a aceptar en su búfer (*window advertisement*), proporcionando el mecanismo de control de flujo extremo a extremo.
 - **Suma de Comprobación (CHECKSUM)**:
 - Campo de **16 bits** utilizado para verificar la integridad del encabezado y de los datos mediante el algoritmo de complemento a uno.
 - **Puntero de Urgencia (URGENT POINTER)**:
 - Campo de **16 bits** que, cuando el bit URG está activo, indica la posición dentro del segmento donde finalizan los datos urgentes enviadamente fuera de banda.
-

2. Opciones de TCP (OPTIONS) y Relleno (PADDING)

El campo de opciones es opcional y de longitud variable. Si las opciones especificadas no ocupan un múltiplo exacto de 32 bits, se añade un campo de **relleno (PADDING)** con ceros para alinear el encabezado a fronteras de 32 bits. Las opciones clave descritas por las fuentes incluyen:

1. Tamaño Máximo de Segmento (MSS - Maximum Segment Size):

- Negociado durante el inicio de la conexión para que el receptor especifique el segmento más grande que puede aceptar en su búfer, permitiendo ajustar los segmentos para que coincidan con la MTU de la red y evitar la fragmentación IP.

2. Escalado de Ventana (Window Scale):

- Consta de 3 octetos y especifica un factor de desplazamiento binario ((S)) para desplazar el campo WINDOW hacia la izquierda, permitiendo superar el límite original de 64 KB y soportar ventanas de varios megabytes en enlaces de alto producto retardo-ancho de banda (*long fat pipes*).

3. Marca de Tiempo (Timestamp):

- Incluye la hora del reloj del emisor y una respuesta de eco para calcular con alta precisión el tiempo de ida y vuelta (RTT) y evitar problemas cuando los números de secuencia se envuelven (PAWS).

4. Confirmación Selectiva (SACK - Selective ACK):

- Permite al receptor informar explícitamente sobre hasta 4 bloques no contiguos de datos recibidos con éxito, permitiendo que el emisor retransmita únicamente los segmentos faltantes en lugar de toda la ventana.
-

3. Verificación de Integridad y el Pseudo-encabezado

Para calcular el campo **CHECKSUM**, el software TCP no evalúa únicamente el segmento, sino que antepone conceptualmente en memoria un **pseudo-encabezado**:

- **Estructura:** Ocupa **12 octetos en IPv4** y **40 octetos en IPv6**. Contiene las direcciones IP de origen y destino, el código de protocolo (valor 6 para TCP) y la longitud total del segmento TCP (TCP LENGTH).
 - **Propósito:** El pseudo-encabezado no se transmite en la red. Sirve exclusivamente para que el receptor verifique que el segmento ha llegado intacto tanto a la computadora correcta como al punto final de conexión especificado (Dirección IP, Puerto TCP).
-

Discuss what these sources say about Congestion Control (AIMD), in the larger context of Transmission Control Protocol (TCP).

En el contexto del **Protocolo de Control de Transmisión (TCP)**, los mecanismos de control de flujo (mediante la ventana anunciada del receptor) evitan que un emisor desborde la capacidad de procesamiento de la máquina de destino. Sin embargo, la infraestructura de la red intermedia —compuesta por enrutadores y enlaces de capacidad finita— también necesita protección frente a la sobrecarga de tráfico.

Cuando múltiples conexiones inyectan más datagramas de los que los enrutadores intermedios pueden procesar, los búferes se llenan y los paquetes son descartados. Si el emisor reacciona retransmitiendo agresivamente, se desencadena una espiral de retransmisiones y retrasos conocida como **colapso por congestión (congestion collapse)**. Para prevenir esta degradación, TCP aplica la estrategia **AIMD (Additive Increase, Multiplicative Decrease / Incremento Aditivo, Disminución Multiplicativa)**.

1. La Ventana de Congestión (congestion_window)

Para regular la cantidad de tráfico que puede inyectarse en la red, TCP gestiona una variable interna denominada **ventana de congestión** (congestion_window o cwnd). La ventana efectiva permitida para transmitir (*Allowed window*) se calcula siempre como el mínimo entre el anuncio de espacio libre del receptor y la ventana impuesta por el estado de congestión de la red:

$$[\text{Allowed_window} = \min(\text{receiver_advertisement}, \text{congestion_window})].$$

2. El Paradigma AIMD: Mecánica Operativa

El algoritmo **AIMD** define el comportamiento dinámico con el que TCP expande o contrae la ventana de congestión en función de la presencia o ausencia de pérdidas de paquetes en la red:

- **Disminución Multiplicativa (*Multiplicative Decrease*):**

- **Detención de congestión:** TCP asume que la mayor parte de las pérdidas de paquetes en una red se deben a congestión en los enrutadores intermedios.
 - **Acción:** Ante la pérdida de un segmento, TCP reduce inmediatamente la ventana de congestión a la mitad (sin reducirla nunca a menos de un segmento).
 - **Justificación:** Proporciona una reducción drástica e inmediata del tráfico inyectado para dar tiempo a que las colas de los enrutadores se vacíen. Si las pérdidas persisten, la ventana se reduce exponencialmente de forma repetida.
- **Incremento Aditivo (*Additive Increase / Evitación de Congestión*):**
 - **Sondeo prudente de ancho de banda:** Una vez que se recupera del estado de congestión y supera el umbral de inicio lento (*ssthresh*), TCP entra en la fase de **evitación de congestión** (*congestion avoidance*).
 - **Acción:** En esta fase, la ventana de congestión incrementa linealmente en **1 segmento** únicamente cuando **todos los segmentos** pertenecientes a la ventana permitida actual han sido confirmados por el receptor.
 - **Ecuación general:** Matemáticamente, ante la llegada de un ACK, el ajuste de la ventana (w) sigue la relación ($w \leftarrow w + \frac{b}{w}$) (donde el estándar usa ($b = 1$) y ($a = 0.5$) para la disminución ($w \leftarrow w - aw$)).
-

3. Fases Complementarias: Inicio Lento (*Slow-Start*) y Retransmisión Rápida

El esquema AIMD opera en conjunto con otras técnicas para optimizar el rendimiento y la estabilidad:

1. Inicio Lento (*Slow-Start*):

- Al iniciar una nueva conexión o al recuperarse tras una congestión severa, TCP asigna $cwnd = 1$ segmento.
- En lugar de escalar duplicando directamente, incrementa $cwnd$ en 1 segmento por cada ACK recibido, lo que se traduce en un crecimiento exponencial (2^k) segmentos por tiempo de ida y vuelta RTT).
- Cuando $cwnd$ alcanza la mitad del tamaño que tenía antes de la congestión, se activa la restricción de incremento aditivo para evitar sobresaturar la red.

2. Retransmisión Rápida y Recuperación Rápida (Reno / NewReno):

- La llegada de **3 ACKs duplicados** se interpreta como pérdida de un segmento aislado sin llegar a agotar el temporizador.
 - En lugar de reiniciar la ventana a 1 segmento (como en el algoritmo original Tahoe), TCP Reno aplica la disminución multiplicativa reduciendo $cwnd$ a la mitad, retransmite el segmento faltante de inmediato y mantiene la transmisión fluyendo mediante inflación artificial de la ventana. NewReno perfecciona este proceso gestionando adecuadamente pérdidas múltiples dentro de la misma ventana.
-

4. Interacción con la Política de Descarte en Enrutadores

El éxito del esquema AIMD de TCP depende en gran medida del comportamiento de las colas en los enrutadores de red:

- **Efecto de *Tail-Drop* y Sincronización Global:** La política tradicional de descarte al final de la cola (*tail-drop*) provoca que, cuando un búfer se llena, se descarten segmentos pertenecientes a múltiples conexiones simultáneamente. Esto lleva a una **sincronización global**, forzando a todas las conexiones TCP concurrentes a entrar en Inicio Lento al mismo tiempo y provocando oscilaciones severas en el tráfico.
 - **Detección Temprana Aleatoria (RED - *Random Early Detection*):** Para evitar la sincronización global, los enrutadores utilizan el algoritmo RED, el cual monitorea un promedio ponderado exponencial del tamaño de la cola (avg) y descarta paquetes de forma aleatoria y probabilística antes de que la cola se llene por completo. Esto induce a conexiones TCP individuales a reducir su ventana mediante AIMD de forma desincronizada y suave.
 - **Notificación Explícita de Congestión (ECN):** Permite a los enrutadores marcar bits de congestión en el encabezado IP/TCP para indicar explícitamente el estado de sobrecarga al emisor sin necesidad de descartar paquetes.
-

Discuss what these sources say about Connection Management, in the larger context of Transmission Control Protocol (TCP).

La **Gestión de Conexiones** en el Protocolo de Control de Transmisión (TCP) abarca el conjunto de procedimientos y estados que permiten a dos aplicaciones negociar el inicio de una sesión, sincronizar parámetros de control, mantener la comunicación bidireccional y finalizar la transferencia de forma ordenada o abrupta sobre una infraestructura IP subyacente no confiable.

1. Puntos Finales y Abstracción de Conexión

- **Definición de Punto Final (*Endpoint*):** TCP define un punto final de comunicación como el par formado por (**Dirección IP, Puerto TCP**).
 - **Identificación de la Conexión:** A diferencia de protocolos como UDP (donde los puertos se asocian a colas individuales de mensajes), la abstracción fundamental de TCP es la **conexión**, identificada por la tupla de cuatro elementos: (**IP Origen, Puerto Origen, IP Destino, Puerto Destino**).
 - **Reutilización de Puertos:** Debido a que TCP identifica las conexiones mediante el par de puntos finales, **un mismo número de puerto TCP local puede ser compartido por múltiples conexiones simultáneas** en un mismo host sin generar ambigüedad.
-

2. Aperturas Pasiva y Activa

Antes de transmitir datos, ambos extremos deben expresar formalmente su acuerdo para conectarse:

- **Apertura Pasiva (*Passive Open*):** Un programa de aplicación (normalmente un servidor) notifica al sistema operativo local que está listo para aceptar peticiones de conexión en un número de puerto específico.
 - **Apertura Activa (*Active Open*):** El programa en el extremo remoto (el cliente) solicita explícitamente el inicio y establecimiento de la conexión hacia dicho puerto.
-

3. Establecimiento de la Conexión (*Three-Way Handshake*)

El inicio de la conexión se realiza mediante un intercambio de tres pasos conocido como **Three-Way Handshake**:

1. **Paso 1 (SYN)**: El cliente (iniciador activo) transmite un segmento con el bit de código **SYN** activado y propone su número de secuencia inicial aleatorio (x) (seq= x).
2. **Paso 2 (SYN + ACK)**: El servidor recibe el SYN, registra el número de secuencia (x) y responde enviando un segmento con los bits **SYN** y **ACK** activos. En él incluye su propio número de secuencia inicial (y) (seq= y) y confirma la recepción de (x) solicitando el siguiente octeto (ACK $x+1$).
3. **Paso 3 (ACK)**: El cliente responde con un segmento **ACK** que confirma el número de secuencia del servidor (ACK $y+1$).

Propósitos del Handshake:

- Verificar que ambas aplicaciones están listas para intercambiar información.
 - Acordar y sincronizar los **Números de Secuencia Iniciales (ISN)** para ambas direcciones del flujo.
 - Tolerancia a fallos de la red IP: evita la creación de conexiones fantasma causadas por solicitudes de conexión duplicadas o retrasadas en la red.
-

4. Cierre Ordenado de la Conexión (*Graceful Close*)

Dado que TCP proporciona un servicio **Full Duplex** (compuesto por dos flujos continuos e independientes de octetos), la terminación de la conexión requiere cerrar ambas direcciones por separado:

1. Cuando una aplicación finaliza el envío de datos, solicita el cierre y TCP envía un segmento con el bit **FIN** activo (seq= x).
 2. El extremo receptor responde de inmediato con un **ACK $x+1$** para detener la retransmisión del FIN e informa a su aplicación local.
 3. La conexión permanece en modo *Half Duplex*: la aplicación que recibió el FIN puede continuar transmitiendo datos en la dirección opuesta si lo necesita.
 4. Cuando la segunda aplicación concluye su transmisión, envía su propio segmento **FIN** (seq= y , ACK $x+1$).
 5. El primer emisor responde con el **ACK $y+1$** definitivo. Una vez cerradas ambas direcciones, ambos puntos finales liberan los registros de la conexión.
-

5. Reinicio Abrupto (*Reset - Bit RST*)

- **Cancelación Instantánea**: Ante condiciones anómalas (por ejemplo, caída de una aplicación, colapso de la memoria o intentos de conexión no autorizados), un punto final puede abortar la conexión enviando un segmento con el bit **RST (RESET)** activo.
 - **Efecto**: A diferencia del cierre ordenado con FIN, el mensaje de reinicio es instantáneo, no se confirma con ACK y destruye la conexión inmediatamente, liberando buffers y notificando la interrupción a la aplicación.
-

6. Máquina de Estados Finitos y el Estado TIMED WAIT

TCP gestiona las transiciones del ciclo de vida de la conexión mediante un modelo formal de **máquina de estados finitos** (que transita por estados como CLOSED, LISTEN, SYN SENT, SYN RECV, ESTABLISHED, FIN WAIT-1, FIN WAIT-2, CLOSE WAIT, CLOSING, LAST ACK y TIMED WAIT).

- **El Estado TIMED WAIT:** Tras enviar la confirmación del último FIN, el extremo que inició el cierre no pasa inmediatamente a CLOSED, sino que permanece en el estado TIMED WAIT durante un periodo de $(2 \times \text{MSL})$ (*Maximum Segment Lifetime* / Tiempo de vida máximo de un segmento).
 - **Funciones del TIMED WAIT:**
 1. Garantizar que los segmentos retrasados o duplicados pertenecientes a la conexión antigua expiren y desaparezcan por completo de la red antes de permitir abrir una nueva sesión con la misma tupla de puntos finales.
 2. Ofrecer un margen para retransmitir la confirmaciones final (ACK) en caso de que esta se haya perdido en la red y el otro extremo vuelva a enviar su segmento FIN.
-

Discuss what these sources say about Network Interaction & Router Policies, in the larger context of Transmission Control Protocol (TCP).

En la arquitectura TCP/IP, el principio de **separación en capas** establece que los enrutadores intermedios operen en la Capa de Red (Capa 3) de manera agnóstica a los estados de las conexiones de la Capa de Transporte. Sin embargo, la forma en que los enrutadores gestionan sus colas internas y reaccionan ante la saturación de tráfico afecta directamente el comportamiento, la estimación de tiempos de espera y el control de congestión de **TCP**.

1. El Aislamiento de Capas y sus Efectos en TCP

- **Procesamiento libre de estado (*Stateless*) en routers:** Los enrutadores reenvían datagramas IP individualmente bajo el paradigma *Best-Effort*, sin mantener registros de las conexiones TCP activas ni asumir responsabilidades de retransmisión.
 - **Impacto del retardo de colas en los temporizadores:** Debido a que TCP mide continuamente el tiempo de ida y vuelta (RTT) para ajustar adaptativamente sus temporizadores de retransmisión, las variaciones drásticas en las colas de los enrutadores pueden provocar la expiración prematura de los temporizadores (*timeouts*). Si el retardo supera el tiempo de espera, TCP asume erróneamente que ocurrió congestión y reduce de inmediato su tasa de envío.
-

2. Política de Descarte por la Cola (*Tail-Drop*) y Sincronización Global

- **Mecánica de *Tail-Drop*:** Los primeros enrutadores empleaban la política de descarte *tail-drop*, la cual añade datagramas a la cola en memoria hasta que esta se llena por completo; a partir de ese instante, descarta todos los datagramas entrantes subsiguientes (la "cola" de la ráfaga).
- **Sincronización Global de Conexiones TCP:** Cuando un enrutador congestionado gestiona tráfico multiplexado de múltiples sesiones TCP simultáneas, *tail-drop* provoca la pérdida simultánea de paquetes pertenecientes a varias conexiones distintas.

- **Efecto destructivo en la red:** Como consecuencia, múltiples instancias de TCP entran al mismo tiempo en la fase de **Inicio Lento (*Slow-Start*)**. Esto induce oscilaciones masivas en la red: fases de colapso extremo de tráfico seguidas de ráfagas simultáneas que vuelven a saturar las colas de los routers.
-

3. Detección Temprana Aleatoria (*RED - Random Early Detection*)

Para erradicar el problema de la sincronización global, los estándares recomiendan el uso del algoritmo de **Detección Temprana Aleatoria (RED)** en los enrutadores.

- **Monitoreo con Promedio Ponderado Exponencial:** En lugar de reaccionar al tamaño instantáneo de la cola, RED calcula un promedio ponderado (*avg*) del tamaño de la cola para permanecer inmune a ráfagas cortas e inofensivas de tráfico.
 - **Mecanismo probabilístico de tres reglas:**
 1. **Si ($avg < T_{min}$):** El datagrama se ingresa a la cola sin riesgo de descarte.
 2. **Si ($avg > T_{max}$):** Todos los datagramas entrantes son descartados (comportamiento *tail-drop*).
 3. **Si ($T_{min} \leq avg \leq T_{max}$):** RED descarta el datagrama entrante de forma aleatoria con una probabilidad (*p*) que escala linealmente según el nivel de ocupación de la cola.
 - **Desincronización de fuentes:** Al descartar paquetes de forma anticipada y aleatoria, RED obliga a conexiones TCP individuales a ejecutar su algoritmo de **Disminución Multiplicativa (AIMD)** de manera desincronizada, manteniendo la utilización del enlace alta y estable.
 - **Medición en octetos:** Cuando la cola se evalúa en octetos en lugar de datagramas, la probabilidad de descarte es proporcional a la cantidad de datos inyectados. Esto protege a los paquetes pequeños de control (como los **ACKs de TCP**), reduciendo retransmisiones innecesarias.
-

4. Notificación Explícita de Congestión (ECN)

- **Información sin pérdida de paquetes:** El mecanismo **ECN (*Explicit Congestion Notification*)** permite a los enrutadores notificar la presencia de congestión antes de tener que descartar datagramas.
 - **Mecanismo cruzado IP/TCP:**
 1. Cuando un router detecta sobrecarga, marca dos bits en el encabezado IP (provenientes del campo *Type of Service*).
 2. El extremo receptor lee la marca e informa al emisor activando un bit reservado en el encabezado TCP de su siguiente confirmación (ACK).
 3. El emisor reacciona reduciendo preventivamente su ventana de congestión (*congestion_window*), evitando pérdidas y retransmisiones.
-

5. Interacción mediante Mensajes de Control ICMP

Los enrutadores también utilizan el **Protocolo de Mensajes de Control de Internet (ICMP)** para retroalimentar al módulo IP/TCP emisor:

- **Ajuste del MSS mediante Descubrimiento de MTU de Ruta:** Si un segmento TCP excede la MTU del siguiente enlace y la fragmentación está prohibida (o en IPv6), el router descarta el datagrama y envía un mensaje ICMP *Packet Too Big / Destination Unreachable* indicando la MTU del enlace. TCP utiliza esta información para ajustar su **Tamaño Máximo de Segmento (MSS)**, evitando la fragmentación IP.
 - **Notificación de bucles de enrutamiento (*Time Exceeded*):** Si un datagrama entra en un bucle y agota su límite de saltos (TTL), el router lo descarta y notifica a la fuente con un mensaje ICMP *Time Exceeded*.
-